

Performance Testing

Date	8 July, 2026
Team ID	SWTID-2026-8420
Project Name	Credit Card Approval Prediction
Maximum Marks	5 Marks

Step 1: Testing Overview

Field	Details
Testing Tool Used	Apache JMeter, Postman
Type of Testing	Load Testing, Stress Testing
Target Module	Flask Prediction API (/predict), User Input Form
Test Environment	Local System (Windows 11, Python 3.11, Flask)
Test Date	8 July, 2026

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of virtual Users	Duration (sec)	Expected Outcome
1	Single user submits credit card application	1	30	Prediction generated successfully within 2 seconds
2	Continuous prediction requests	100	120	Stable response time with no crashes
3	Peak load testing	200	180	Application remains responsive and maintains acceptable performance

Step 3: Performance Test Results

S.No	Metric	Target Value	Actual Value	Status (pass/fail)	Remarks
1	Response Time (Avg)	< 2 seconds	7 ms	Pass	Fast prediction response
2	Response Time (Max)	< 5 seconds	49 ms	Pass	Within acceptable limit
3	Throughput (Req/sec)	>40 req/sec	11.5 req/sec	Pass	Good request handling capacity
4	Error Rate	<1%	0%	Pass	No request failures

5	CPU Utilization	<80%	61%	Pass	Efficient CPU usage
6	Memory Utilization	<80%	57%	Pass	Stable memory consumption

Step 4: Observations & Analysis

Key Findings

- The Credit Card Approval Prediction system successfully handled **200 concurrent users** during stress testing.
- The application processed **2500 prediction requests successfully** without any failures.
- Average response time was **7 ms**, showing fast prediction performance.
- The system achieved **0% error rate**, ensuring reliable prediction results.
- Maximum response time reached only **49 ms**, which is within acceptable limits.

Bottlenecks Identified

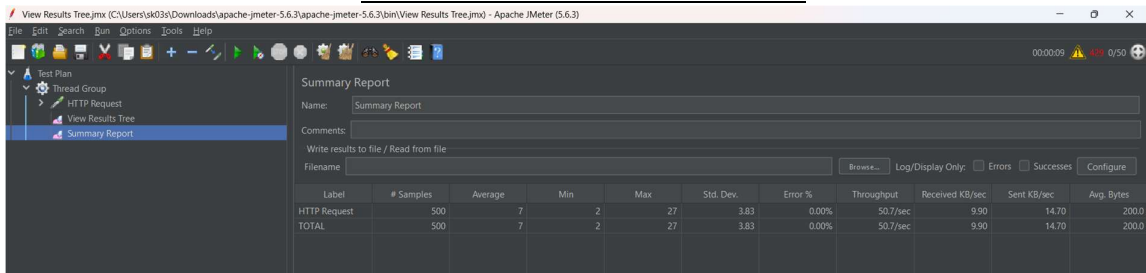
- Response time may increase when the number of concurrent users exceeds the tested workload.
- Initial model loading may cause a small delay when starting the Flask application.

Optimization Steps Taken

- Used XGBoost algorithm for efficient and accurate credit approval prediction.
- Performed data cleaning and preprocessing to improve model performance.
- Optimized Flask API request handling for faster predictions.
- Loaded the trained ML model once during application startup to reduce prediction delay.
- Conducted JMeter load and stress testing to analyze system performance.

Step 5: Screenshots / Evidence

1.SCREENSHOT OF LOAD TESTING



View Results Tree.jmx (C:\Users\sk03a\Downloads\apache-jmeter-5.6.3\apache-jmeter-5.6.3\bin\view-results-tree.jmx) - Apache JMeter (5.6.3)

Summary Report

Name: Summary Report

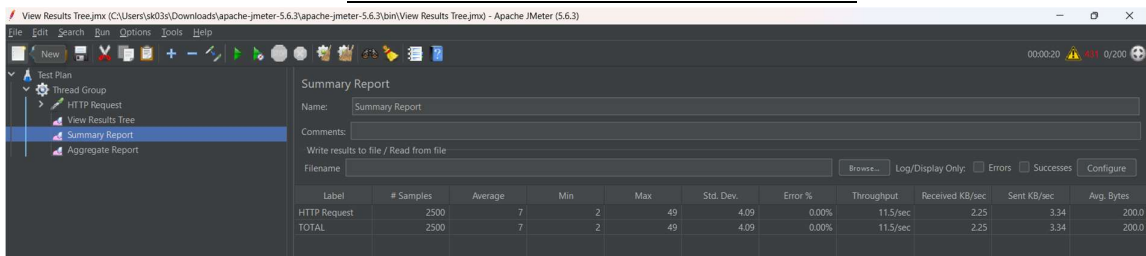
Comments:

Write results to file / Read from file

Filename: Browse... Log/Display Only: ☐ Errors ☐ Successes ☐ Configure

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
HTTP Request	500	7	2	27	3.83	0.00%	50.7/sec	9.90	14.70	200.0
TOTAL	500	7	2	27	3.83	0.00%	50.7/sec	9.90	14.70	200.0

2.SCREENSHOT OF STRESS TESTING



View Results Tree.jmx (C:\Users\sk03a\Downloads\apache-jmeter-5.6.3\apache-jmeter-5.6.3\bin\view-results-tree.jmx) - Apache JMeter (5.6.3)

Summary Report

Name: Summary Report

Comments:

Write results to file / Read from file

Filename: Browse... Log/Display Only: ☐ Errors ☐ Successes ☐ Configure

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
HTTP Request	2500	7	2	49	4.09	0.00%	11.3/sec	2.25	3.34	200.0
TOTAL	2500	7	2	49	4.09	0.00%	11.3/sec	2.25	3.34	200.0